2322ENG: Engineering C

Programming Laboratory 1: Introduction to the C programming

Instructions:

This laboratory is designed to introduce you to the C language, including the IDE and programming environment to be used throughout the course. You will write some simple C programs, and compile them to make executables. This lab session is **not assessed**.

You should have read all "Introduction to C" notes and watched all of the associated videos prior to attending this lab!

PART 1: Installing the software

From either the course website or elsewhere, download the programming environment that you will used for this course. Code::Blocks is recommended, but you may use another environment that is equivalent.

A video is available on the course site showing the steps required to install Code::Blocks and run a simple program, so please follow along with this if you have any difficulties.

If you are using Linux or MacOS (command line), ensure that gcc and other tools are installed:
To ensure that all the necessary compilers, linkers, and other tools are installed, run the following command:

```
sudo apt install build-essential
```

This will install all the necessary tools to create C programs. You can check that gcc is installed by typing "gcc" at the prompt. The response should be:

```
gcc: fatal error: no input files
```

PART 2: Writing your first C program

Typing your first C program

1. Open up your chosen IDE or text editor, and type in the "Hello World" program as written below. Ensure that this is saved correctly as `hello.c`.

```
  nano 2.6.1                    File: hello.c                    Modified

#include <stdio.h>

int main(void)
{
    printf("Hello world\n");

    return (0);
}

                [ line 8/9 (88%), col 2/2 (100%), char 79/80 (98%) ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Uncut Text^T To Spell  ^_ Go To Line
```

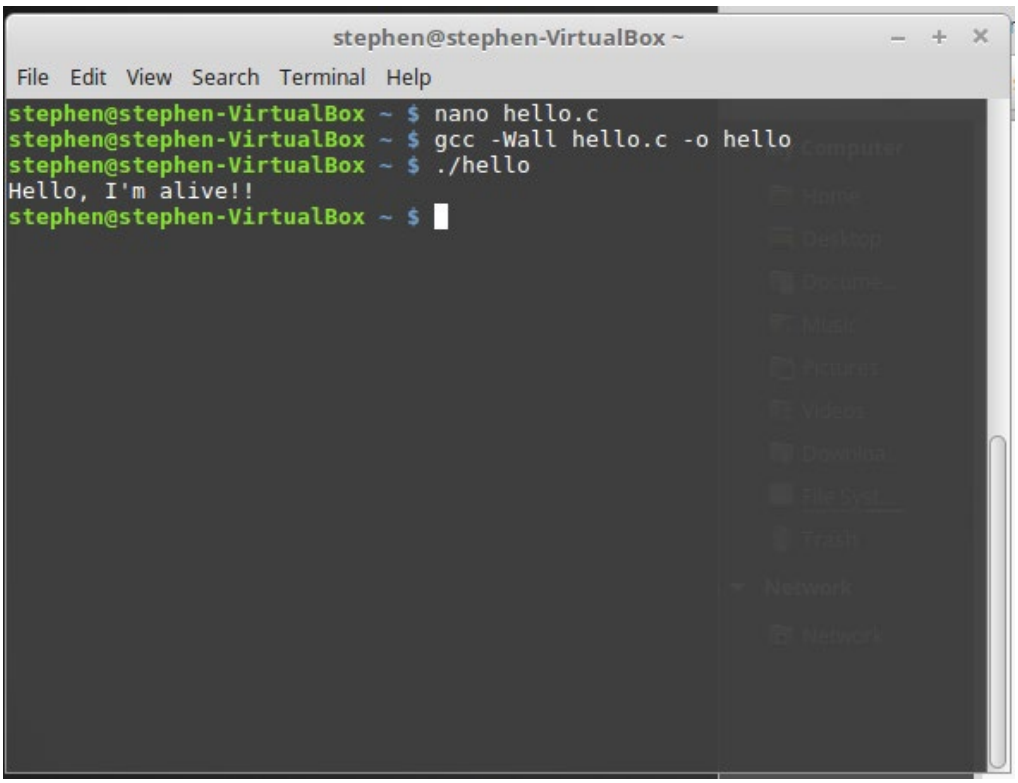
2. Compile the program using either the compile button in your IDE or the following line from the linux prompt:

   `gcc -Wall hello.c -o hello`

   The option '`-o hello`' tells the compiler to give the name 'hello' to the output executable file. The option '`-Wall`' tells the compiler to give all warnings.

   Ensure that there are no compilation or build errors!
   If the gcc command does not work, you may not have installed it properly!

3. Check the contents of the folder in which you saved the hello.c source file. You should now see an object file (.o) and an executable (.exe in windows).
4. Run the program through the IDE or the linux shell. To run the executable from the shell, type: ./hello



Congratulations!! You have written, compiled and executed your very first C program. You can follow this same procedure for all future programs. Remember that when changes are made to the C source file, you need to compile it to an executable first.

Linking the maths library:

As engineers, we need to perform mathematics often in our programs so we need to access the mathematics library that comes with C.

1. Using your chosen editor, create a new file and save it as angle.c
2. Type the following C program in save it. Note the inclusion of the <math.h> line at the top, which instructs the compiler to add in the maths library header when compiling.

```
  nano 2.6.1                          File: angle.c

#include <stdio.h>
#include <math.h>

int main(void)
{
    float angle, s, c, tangent, unity;

    printf("Enter an angle in radians: ")
    scanf("%f", &angle);

    s = sin(angle);
    c = cos(angle);
    tangent = s / c;

    unity = sqrt(s * s + c * c);

    printf("The answers are:\n%f\n%f\n%f\n%f\n", s, c, tangent, unity);

    return (0);
}

                    [ line 20/21 (95%), col 2/2 (100%), char 327/328 (99%) ]
^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^J Justify     ^C Cur Pos
^X Exit        ^R Read File   ^\ Replace     ^U Uncut Text  ^T To Spell    ^_ Go To Line
```

3. Now compile this file in the IDE, or go the terminal window and compile it using the command:

   ```
   gcc -Wall angle.c -o angle
   ```

4. Notice that gcc prints out some errors

   ```
   angle.c: In function ‘main’:
   angle.c:9:5: error: expected ‘;’ before ‘scanf’
        scanf("%f", &angle);
   ```

Recall that every statement in C requires a *semicolon* on the end. This error message is telling us that a semicolon is missing before the scanf on line 9.

5. Add a semicolon ; to the end of the printf statement and try to compile again.
6. We get another set of errors:

   ```
   /tmp/ccvzSxVL.o: In function `main':
   angle.c:(.text+0x39): undefined reference to `sin'
   angle.c:(.text+0x56): undefined reference to `cos'
   angle.c:(.text+0x98): undefined reference to `sqrt'
   collect2: error: ld returned 1 exit status
   ```

These errors are actually coming from the linker. They are telling us that the linker cannot find a definition for the mathematics functions sin(), cos() and sqrt(). Therefore, we need to link in the maths library. **Note: Code::Blocks and some other IDEs may link the math library by default, so you may not receive this error.**

7. Compile again using the following command (`-lm` means link with the maths library). In

Code::Blocks, additional libraries can be added to the compilation process by going to the "Settings" menu, choosing "Compiler Options", ensure that "GCC C Compiler" is selected at the top, and then going to the "Linker Options" tab. Libraries can be added to the list on the left, and they will then be automatically added to the program if required.

```
gcc -Wall angle.c -lm -o angle
```

8. Now the program should compile. Run it by typing `./angle`

```
Enter an angle in radians: 3
The answers are:
0.141120
-0.989992
-0.142547
1.000000
```

Final exercises (these are normally randomly assessed, but not this week!):

1. Write a C program that asks the user to enter an angle in degrees, then converts out that angle in both radians and gradians.

Note: 1 gradian = 9/10 of a degree

Tips:

- The value for π is stored in a macro variable `M_PI` from the `math.h` header file.
- Best to use floating point variables for storing the angles.

Sample output (input from keyboard is underlined):

```
This program will convert an angle in degrees to radians and gradians
Enter the angle: 120
The angle is equal to 2.094395 rad and 133.333328 grad
```

Ensure that you check your results to verify that they are correct!

2. Adapt the program above such that it reads a value in meters from the user, and then converts that into yards, miles, and furlongs (look those up!).

3. Write a C program to calculate the reactance of a capacitor. The user should be prompted to enter the capacitance of the capacitor (in Farads), and the frequency of the signal (in Hz). Your program can then calculate the reactance using the formula $X = 2\pi fC$, where X is the reactance, f is the frequency, and C is the capacitance. All values should be floating points, and the answer should be displayed in appropriate scientific notation. Use an online calculator to check your results are correct.